

JHU AI Engineering

Module 8 - PKI, SSL/TLS, Digital Certificates

Part 1: Intro to OpenSSL toolkit

*Note: I used **docker** to create the VM, not VMWare. Given this configuration, some differences in commands are necessary, but the outputs are more or less the same.*

```
[2026-07-17 09:32:42] [ft@nostromo] [~/Documents/code/jhu/jhu-ai-eng-course/module8/lab]
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
65a4c1c2d637   module8-pki-pki  "sleep infinity"        16 hours ago   Up 8 minutes   pki_lab
```

A screenshot of the running docker container.

*Note: Given the setup using **docker**, we need to use **docker compose exec pki <command>** rather than the direct commands when operating directly inside the container.*

hostnamectl

This command fails given that:

- A container is not a machine
- Dockerized Ubuntu doesn't include **hostnamectl** in the minimal image.

```
[2026-07-17 09:38:40] [ft@nostromo] [~/Documents/code/jhu/jhu-ai-eng-course/module8/lab]
$ docker compose exec pki hostnamectl
OCI runtime exec failed: exec failed: unable to start container process: exec: "hostnamectl": executable file not found in $PATH
```

*A screenshot of the failed **hostnamectl** running inside the container.*

```
[2026-07-17 09:43:49] [ft@nostromo] [~/Documents/code/jhu/jhu-ai-eng-course/module8/lab]
$ hostnamectl
Static hostname: nostromo
    Icon name: computer-laptop
    Chassis: laptop
Chassis Asset Tag: No Asset Tag
    Machine ID: b152d31f257a4ebebc045884a51e6f84
    Boot ID: 28657a83ba874e39b6a9ba78885bc7a0
Operating System: Arch Linux
    Kernel: Linux 7.1.3-arch2-1
    Architecture: x86-64
Hardware Vendor: ASUSTeK COMPUTER INC.
Hardware Model: ROG Zephyrus G14 GA403WR_GA403WR
Hardware Version: 1.0
Firmware Version: GA403WR.309
Firmware Date: Wed 2025-09-17
Firmware Age: 9month 4w 1d
```

*A screenshot of **hostnamectl** running on the native machine.*

openssl version -a

```
[2026-07-17 09:35:16] [ft@nostromo] [~/Documents/code/jhu/jhu-ai-eng-course/module8/lab]
$ docker compose exec pki openssl version -a
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
built on: Tue Jun 2 19:33:25 2026 UTC
platform: debian-amd64
options: bn(64,64)
compiler: gcc -fPIC -pthread -m64 -Wa,--noexecstack -Wall -fzero-call-used-regs=used-gpr -DOPENSSL_TLS_SECURITY_LEVEL=2 -Wa,--noexecstack -g
-O2 -fno-omit-frame-pointer -mno-omit-leaf-frame-pointer -ffile-prefix-map=/build/openssl-gzaf8/openssl-3.0.13=. -fstack-protector-strong -f
stack-clash-protection -Wformat -Werror=format-security -fcf-protection -idebug-prefix-map=/build/openssl-gzaf8/openssl-3.0.13=/usr/src/open
ssl-3.0.13-0ubuntu3.11 -DOPENSSL_USE_NODELETE -DL_ENDIAN -DOPENSSL_PIC -DOPENSSL_BUILDING_OPENSSL -DDEBUG -Wdate-time -D_FORTIFY_SOURCE=3
OPENSSLDIR: "/usr/lib/ssl"
ENGINESDIR: "/usr/lib/x86_64-linux-gnu/engines-3"
MODULESDIR: "/usr/lib/x86_64-linux-gnu/openssl-modules"
Seeding source: os-specific
CPUINFO: OPENSSL_ia32cap=0x7ed8320b078bffff:0x19405fdef1bb97ab
```

A screenshot of the `openssl version -a` run inside the container.

openssl ciphers

```
[2026-07-17 09:45:25] [ft@nostromo] [~/Documents/code/jhu/jhu-ai-eng-course/module8/lab]
$ docker compose exec pki openssl ciphers
TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256:TLS_AES_128_GCM_SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:DHE-RSA-
AES256-GCM-SHA384:ECDSA-CHACHA20-POLY1305:ECDSA-CHACHA20-POLY1305:DHE-RSA-CHACHA20-POLY1305:ECDSA-AES128-GCM-SHA256:ECDSA-AES128-GCM-SHA256:DHE-RSA-AES128-GCM-SHA256:DHE-RSA-AES128-GCM-SHA256:ECDSA-AES256-GCM-SHA384:ECDSA-AES256-GCM-SHA384:DHE-RSA-AES256-GCM-SHA384:DHE-RSA-AES256-GCM-SHA384:ECDSA-AES128-SHA256:ECDSA-AES128-SHA256:DHE-RSA-AES128-SHA256:DHE-RSA-AES128-SHA256:ECDSA-AES256-SHA:ECDSA-AES256-SHA:DHE-RSA-AES256-SHA:DHE-RSA-AES256-SHA:ECDSA-AES128-SHA:ECDSA-AES128-SHA:DHE-RSA-AES128-SHA:DHE-RSA-AES128-SHA:RSA-PSK-AES256-GCM-SHA384:DHE-PSK-AES256-GCM-SHA384:RSA-PSK-CHACHA20-POLY1305:DHE-PSK-CHACHA20-POLY1305:ECDH
-E-PSK-CHACHA20-POLY1305:AES256-GCM-SHA384:PSK-AES256-GCM-SHA384:PSK-CHACHA20-POLY1305:RSA-PSK-AES128-GCM-SHA256:DHE-PSK-AES128-GCM-SHA256:AES
128-GCM-SHA256:PSK-AES128-GCM-SHA256:AES256-SHA256:AES128-SHA256:ECDSA-PSK-AES256-CBC-SHA384:ECDSA-PSK-AES256-CBC-SHA:SRP-RSA-AES-256-CBC-SHA
:SRP-AES-256-CBC-SHA:RSA-PSK-AES256-CBC-SHA384:DHE-PSK-AES256-CBC-SHA384:RSA-PSK-AES256-CBC-SHA:DHE-PSK-AES256-CBC-SHA:AES256-SHA:PSK-AES256-
CBC-SHA384:PSK-AES256-CBC-SHA:ECDSA-PSK-AES128-CBC-SHA256:ECDSA-PSK-AES128-CBC-SHA:SRP-RSA-AES-128-CBC-SHA:SRP-AES-128-CBC-SHA:RSA-PSK-AES128
-CBC-SHA256:DHE-PSK-AES128-CBC-SHA256:RSA-PSK-AES128-CBC-SHA:DHE-PSK-AES128-CBC-SHA:AES128-SHA:PSK-AES128-CBC-SHA256:PSK-AES128-CBC-SHA
```

A screenshot of the `openssl ciphers` run inside the container. [\[1\]](#)

Part 2: Quick warm up on Symmetric Encryption and Decryption

```
[2026-07-17 09:47:37] [ft@nostromo] [~/Documents/code/jhu/jhu-ai-eng-course/module8/lab]
$ docker compose exec pki bash
ubuntu@65a4c1c2d637:/lab$ echo "hello world from -student name-" > plain.txt
ubuntu@65a4c1c2d637:/lab$ ls
plain.txt
ubuntu@65a4c1c2d637:/lab$ openssl enc -aes-256-cbc -pbkdf2 -in plain.txt -out encrypted.bin
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
ubuntu@65a4c1c2d637:/lab$ ls
encrypted.bin plain.txt
ubuntu@65a4c1c2d637:/lab$ openssl enc -aes-256-cbc -d -pbkdf2 -in encrypted.bin
enter AES-256-CBC decryption password:
hello world from -student name-
ubuntu@65a4c1c2d637:/lab$ exit
exit
```

A screenshot of all steps required in Part 2. Demonstrating encrypting and decrypting a text file. [\[2\]\[3\]\[4\]](#)

Part 3: PKI, Working with Public and Private Keys

```
[2026-07-17 11:33:45] [ft@nostromo] [~/Documents/code/jhu/jhu-ai-eng-course/module8/lab]
$ docker compose exec pki bash
ubuntu@65a4c1c2d637:/lab$ ls
encrypted.bin  plain.txt
ubuntu@65a4c1c2d637:/lab$ openssl genrsa -out key.pem 1024
ubuntu@65a4c1c2d637:/lab$ ls
encrypted.bin  key.pem  plain.txt
ubuntu@65a4c1c2d637:/lab$ cat key.pem
-----BEGIN PRIVATE KEY-----
MIICdwIBADANBgkqhkiG9w0BAQEFAASCAmEwgGJdAgEAAoGBAKwXOYYmjINoHPVK
KHbnJXzSc1VA+ds589hWvcc3S/dKjMoQDcSFxQFKeJa7001eK6db6pv3N1TuPUNo
6bU+6u3sgCrGLs31pR7BG0p+4qqZTD+bNtKn4EEA0AGzxjbjNNHWNwfaD0UhlqUS
bLb3t5A8I1Zr/qWUtcJ9VuYxoZjXAgMBAAECgYBR8p1J7IFs8d8YT0AFnvyS/AQ/
zLkuU+JyWpv0ibh7Ad0Z05vSwYCwpmfXv0D2DqnADiIwIwE0SIST3RmtXVCxQ8ec
1lCTH+FPtHh53ajNMTTrDWiKruT2L0G/KUQQpGD+SM69eU3MWNcsGiw7UWJcW8Ji
q4SPjU7/5AppL7P0gQJBANv+nHE8MrEnroJJcAHsol2XSG8oy28y6freKTbQQFVr
CXqh8NQ1ogjN7uw+nqdCkySbCRHgQaAhS7wdexeNxa8CQQD0WlevEq9YRa1KfXY8
49SzmKjVg0Amr4eZb2nW0BSjorDVmQLJmHVqR3ZAVD2HzBco/q96NxsIFQtvLNdd
TNFZAkEathj/IQpihn49N72hvtJyjveJfd7C3j8EGclCSBGKvhDFDgdwneB9K5tT
Dcg/1JSRYuil7MxlzUp5CaPvq0Bw2wJBAJkCLjBX54LJrJzaaNj/itFeHpGLT/8f
eqmQDmfFRPiNB+pFmeaH0NBsisdqb3GJKMcyCkXSJI4apK7cDhlVg4kCQC6pbu3Z
zWAR1mDrUbptDPL4TkKz/aRuth+TyJjVuwJA06yX3zHvIiBTjh9t8fIk1/+Udc8g
EgykOgr1Nb8rosY=
-----END PRIVATE KEY-----
```

Segment 1 of Part 3. Generating a key and displaying the value. Insecure! [\[5\]](#)

```

ubuntu@65a4c1c2d637:/lab$ openssl rsa -in key.pem -text -noout
Private-Key: (1024 bit, 2 primes)
modulus:
    00:ac:17:39:86:26:8c:83:68:1c:f5:4a:28:76:e7:
    25:7c:d2:73:55:40:f9:db:39:f3:d8:56:bd:c7:37:
    4b:f7:4a:8c:ca:10:0d:c4:85:c5:01:4a:78:96:bb:
    3b:4d:5e:2b:a7:5b:ea:9b:f7:37:54:ee:3d:43:68:
    e9:b5:3e:ea:ed:ec:80:2a:c6:2e:cd:f5:a5:1e:c1:
    18:ea:7e:e2:aa:99:4c:3f:9b:36:d2:a7:e0:41:00:
    38:01:b3:c6:36:e3:34:d1:d6:37:07:da:0f:45:21:
    96:a5:12:6c:b6:f7:b7:90:3c:22:56:6b:fe:a5:94:
    b5:c2:7d:56:e6:31:a1:98:d7
publicExponent: 65537 (0x10001)
privateExponent:
    51:f2:9d:49:ec:81:6c:f1:df:18:4f:40:05:9e:fc:
    92:fc:04:3f:cc:b9:2e:53:e2:72:5a:9b:f4:89:b8:
    7b:01:dd:19:3b:9b:d2:c1:80:96:a4:c7:d7:bf:40:
    f6:0e:a9:eb:00:32:30:23:01:0e:48:84:93:dd:19:
    ad:5d:50:b1:43:c7:9c:96:50:93:1f:e1:4f:b4:78:
    79:dd:a8:cd:31:3a:c3:5a:22:ab:b9:3d:8b:d0:6f:
    ca:51:04:29:18:3f:92:33:af:5e:53:73:16:35:cb:
    06:8a:85:bb:51:62:5c:5b:c2:62:ab:84:8f:8d:4e:
    ff:e4:0a:69:2f:b3:f4:81
prime1:
    00:d5:7e:9c:71:3c:32:b1:27:ae:82:49:70:01:ec:
    a2:5d:97:48:6f:28:cb:6f:32:e9:fa:de:29:36:d0:
    40:55:6b:09:7a:a1:f0:d4:35:a2:08:cd:ee:ec:3e:
    9e:a7:42:93:24:9b:09:11:e0:41:a0:21:4b:bc:1d:
    7b:17:8d:c5:af
prime2:
    00:ce:5a:57:af:12:af:58:45:a9:4a:7d:76:3c:e3:
    d4:b3:9a:48:d5:83:40:26:af:87:99:6f:69:d6:d0:
    14:a3:a2:b7:55:99:02:c9:98:75:6a:47:76:40:54:
    3d:87:cc:17:28:fe:af:7a:37:1b:08:15:0b:6f:2c:
    d7:5d:4c:d1:59
exponent1:
    00:b6:18:ff:21:0a:62:86:7e:3d:37:bd:a1:be:d2:
    72:8e:f7:89:7d:de:c2:de:3f:04:19:c9:42:48:11:
    8a:be:10:c5:0e:07:70:9d:e0:7d:2b:9b:53:0d:c8:
    3f:94:94:91:62:e8:a5:ec:cc:65:cd:4a:79:09:a3:
    ef:a8:e0:70:db
exponent2:
    00:99:02:2e:30:57:e7:82:c9:ac:9c:da:68:d8:ff:
    8a:d1:5e:1e:91:8b:4f:ff:1f:7a:a9:90:0e:67:c5:
    44:f8:8d:07:ea:45:99:e6:87:d0:d0:6c:8a:c7:6a:
    6f:71:89:28:c7:32:0a:45:d2:24:8e:1a:a4:ae:dc:
    0e:19:55:83:89
coefficient:
    2e:a9:6e:ed:d9:cd:60:11:d6:60:eb:51:ba:6d:0c:
    f2:f8:4e:42:b3:fd:a4:6e:b6:1f:93:c8:98:d5:bb:
    02:40:d3:ac:97:df:31:ef:22:20:53:8e:1f:6d:f1:
    f2:24:97:ff:94:75:cf:20:12:0c:a4:3a:0a:f5:35:
    bf:2b:a2:c6
ubuntu@65a4c1c2d637:/lab$ openssl rsa -in key.pem -des3 -out enc-key.pem
writing RSA key
Enter pass phrase:
Verifying - Enter pass phrase:
ubuntu@65a4c1c2d637:/lab$ openssl rsa -in key.pem -pubout -out pub-key.pem
writing RSA key
ubuntu@65a4c1c2d637:/lab$ ls
enc-key.pem  encrypted.bin  key.pem  plain.txt  pub-key.pem

```

Next segment of Part 3. Displaying the key values in hex and encrypting the key. [\[6\]](#)

```

ubuntu@65a4c1c2d637:/lab$ echo "hello world from -pbk-" > plain2.txt
ubuntu@65a4c1c2d637:/lab$ openssl pkcs8 -in plain2.txt -inkey key.pem -out encrypted.bin
ubuntu@65a4c1c2d637:/lab$ openssl pkcs8 -in encrypted.bin -inkey key.pem -out plain2_decrypted.txt
cat: plain2_decrypted.txt: No such file or directory
ubuntu@65a4c1c2d637:/lab$ openssl pkcs8 -in plain2.txt -inkey key.pem -out longencrypted.bin
ubuntu@65a4c1c2d637:/lab$ openssl enc -aes-256-cbc -pbkdf2 -in toolong.txt -out longencrypted.bin
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
ubuntu@65a4c1c2d637:/lab$ echo "ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff" > toolong.txt
ubuntu@65a4c1c2d637:/lab$ openssl enc -aes-256-cbc -pbkdf2 -in toolong.txt -out longencrypted.bin
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
ubuntu@65a4c1c2d637:/lab$ openssl pkcs8 -in toolong.txt -inkey key.pem -out longencrypted.bin
public key operation error
407AC39C7F0000:error:8200006E:rsa routines:ossl_rsa_padding_add_PKCS1_type_2:ex:data too large for key size:../crypto/rsa/rsa_pk1.c:123:
ubuntu@65a4c1c2d637:/lab$

```

A demonstration of overflowing the RSA character limit, including a fumbled command! [\[7\]](#)

Part 3.5: Self-signing a certificate

```
[2026-07-17 12:51:47] [ft@nostromo] [~/Documents/code/jhu/jhu-ai-eng-course/module8/lab]
$ docker compose exec pki bash
ubuntu@65a4c1c2d637:/lab$ openssl req -new -key key.pem -out cert.csr -subj "/C=US/ST=NC/L=CHARLOTTE/O=JHU/CN=Fred Teumer"
ubuntu@65a4c1c2d637:/lab$ openssl x509 -req -days 365 -in cert.csr -signkey key.pem -out cert.pem
Certificate Request self-signature ok
subject=C = US, ST = NC, L = CHARLOTTE, O = JHU, CN = Fred Teumer
ubuntu@65a4c1c2d637:/lab$ openssl x509 -in cert.pem -text -noout
Certificate:
    Data:
        Version: 1 (0x0)
        Serial Number:
            2e:d6:a6:c8:0a:fe:ae:bc:54:28:e5:78:79:3f:c5:4d:ec:de:0b:79
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C = US, ST = NC, L = CHARLOTTE, O = JHU, CN = Fred Teumer
        Validity
            Not Before: Jul 17 16:52:12 2026 GMT
            Not After : Jul 17 16:52:12 2027 GMT
        Subject: C = US, ST = NC, L = CHARLOTTE, O = JHU, CN = Fred Teumer
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (1024 bit)
            Modulus:
                00:ac:17:39:86:26:8c:83:68:1c:f5:4a:28:76:e7:
                25:7c:d2:73:55:40:f9:db:39:f3:d8:56:bd:c7:37:
                4b:f7:4a:8c:ca:10:0d:c4:85:c5:01:4a:78:96:bb:
                3b:4d:5e:2b:a7:5b:ea:9b:f7:37:54:ee:3d:43:68:
                e9:b5:3e:ea:ed:ec:80:2a:c6:2e:cd:f5:a5:1e:c1:
                18:ea:7e:e2:aa:99:4c:3f:9b:36:d2:a7:e0:41:00:
                38:01:b3:c6:36:e3:34:d1:d6:37:07:da:0f:45:21:
                96:a5:12:6c:b6:f7:b7:90:3c:22:56:6b:fe:a5:94:
                b5:c2:7d:56:e6:31:a1:98:d7
            Exponent: 65537 (0x10001)
        Signature Algorithm: sha256WithRSAEncryption
        Signature Value:
            55:8d:71:1b:09:c0:b1:5a:1b:95:75:0c:28:c3:6d:72:18:a8:
            df:b0:ee:38:d2:dd:c6:8d:82:e4:db:d6:f3:10:80:1a:35:63:
            60:cc:e2:af:30:4c:ae:27:34:b0:68:24:37:8e:ec:e3:f7:af:
            fd:6b:36:e2:a9:1c:f3:f6:43:f0:21:57:49:c0:a0:04:f4:42:
            18:c1:e6:3f:7b:8c:97:9a:b3:8b:1f:dd:ed:9b:8a:a5:83:6e:
            20:15:e5:db:0d:f2:fc:37:5a:0e:c7:5b:af:2c:58:1a:9e:da:
            ad:88:d5:be:b9:a3:a2:47:56:dd:10:1a:07:ba:1e:8a:12:2a:
            c8:d7
ubuntu@65a4c1c2d637:/lab$
```

Creating and signing a certificate. [\[9\]](#)[\[10\]](#)[\[11\]](#)

Part 4: Use PKI to securely interact with fellow students, post results

```
[2026-07-17 11:55:47] [ft@nostrono] [~/Documents/code/jhu/jhu-ai-eng-course/module8/lab]
$ docker compose exec pki bash
ubuntu@65a4c1c2d637:/lab$ ls
enc-key.pem  encrypted.bin  key.pem  longencrypted.bin  plain.txt  plain2.txt  plain2_decrypted.txt  pub-key.pem  toolong.txt
ubuntu@65a4c1c2d637:/lab$ openssl genrsa -out fellow.pem 1024
ubuntu@65a4c1c2d637:/lab$ ls
enc-key.pem  encrypted.bin  fellow.pem  key.pem  longencrypted.bin  plain.txt  plain2.txt  plain2_decrypted.txt  pub-key.pem  toolong.txt
ubuntu@65a4c1c2d637:/lab$ openssl rsa -in fellow.pem -pubout -out pub-fellow.pem
writing RSA key
ubuntu@65a4c1c2d637:/lab$ ls
enc-key.pem  encrypted.bin  fellow.pem  key.pem  longencrypted.bin  plain.txt  plain2.txt  plain2_decrypted.txt  pub-fellow.pem  pub-key.pem  toolong.txt
ubuntu@65a4c1c2d637:/lab$ vi fred.txt
bash: vi: command not found
ubuntu@65a4c1c2d637:/lab$ echo -e "Fred Teumer\nSteak and Waffles\nRally House Bops" > fred.txt
ubuntu@65a4c1c2d637:/lab$ cat fred.txt
Fred Teumer
Steak and Waffles
Rally House Bops
ubuntu@65a4c1c2d637:/lab$ openssl pkeyutl -encrypt -inkey fellow.pem -pubin -in fred.txt -out encryptedname.enc
Could not read public key from fellow.pem
404786A4297F0000:error:1608010C:STORE routines:ossl_store_handle_load_result:unsupported:../crypto/store/store_result.c:151:
pkeyutl: Error initializing context
ubuntu@65a4c1c2d637:/lab$ openssl rsa -in fellow.pem -des3 -out enc-fellow.pem
writing RSA key
Enter pass phrase:
Verifying - Enter pass phrase:
Verify failure
unable to write key
4057D7DD137F0000:error:1400006B:UI routines:UI_process:processing error:../crypto/ui/ui_lib.c:548:while reading strings
4057D7DD137F0000:error:07800028:common libcrypto routines:do_ui_passphrase:UI lib:../crypto/passphrase.c:187:
4057D7DD137F0000:error:1C80009F:Provider routines:p8info_to_encp8:unable to get passphrase:../providers/implementations/encode_decode/encode_key2any.c:116:
ubuntu@65a4c1c2d637:/lab$ openssl pkeyutl -encrypt -inkey pub-fellow.pem -pubin -in fred.txt -out encryptedname.enc
ubuntu@65a4c1c2d637:/lab$ openssl pkeyutl -decrypt -inkey fellow.pem -in encryptedname.enc -out decryptedname.txt
ubuntu@65a4c1c2d637:/lab$ cat decryptedname.txt
Fred Teumer
Steak and Waffles
Rally House Bops
ubuntu@65a4c1c2d637:/lab$
```

Simulating using a separate public key to encrypt the file, decrypting with a different private key. [8]

References

1. OpenSSL Project. "openssl-ciphers — SSL cipher display and cipher-list utility." OpenSSL 3.0 Manual. <https://docs.openssl.org/3.0/man1/openssl-ciphers/> (accessed July 17, 2026).
2. OpenSSL Project. "openssl-enc — symmetric cipher routines." OpenSSL 3.0 Manual. <https://docs.openssl.org/3.0/man1/openssl-enc/> (accessed July 17, 2026).
3. National Institute of Standards and Technology. "Advanced Encryption Standard (AES)." FIPS 197. <https://csrc.nist.gov/pubs/fips/197/final> (accessed July 17, 2026).
4. Moriarty, K., et al. "PKCS #5: Password-Based Cryptography Specification Version 2.1." RFC 8018, IETF. <https://www.rfc-editor.org/rfc/rfc8018> (accessed July 17, 2026).
5. OpenSSL Project. "openssl-genrsa — generate an RSA private key." OpenSSL 3.0 Manual. <https://docs.openssl.org/3.0/man1/openssl-genrsa/> (accessed July 17, 2026).
6. OpenSSL Project. "openssl-rsa — RSA key processing command." OpenSSL 3.0 Manual. <https://docs.openssl.org/3.0/man1/openssl-rsa/> (accessed July 17, 2026).
7. Moriarty, K., et al. "PKCS #1: RSA Cryptography Specifications Version 2.2." RFC 8017, IETF. <https://www.rfc-editor.org/rfc/rfc8017> (accessed July 17, 2026).
8. OpenSSL Project. "openssl-pkeyutl — public-key algorithm command." OpenSSL 3.0 Manual. <https://docs.openssl.org/3.0/man1/openssl-pkeyutl/> (accessed July 17, 2026).
9. OpenSSL Project. "openssl-req — PKCS#10 certificate request and generating utility." OpenSSL 3.0 Manual. <https://docs.openssl.org/3.0/man1/openssl-req/> (accessed July 17, 2026).
10. OpenSSL Project. "openssl-x509 — certificate display and signing utility." OpenSSL 3.0 Manual. <https://docs.openssl.org/3.0/man1/openssl-x509/> (accessed July 17, 2026).
11. Cooper, D., et al. "Internet X.509 Public Key Infrastructure Certificate and CRL Profile." RFC 5280, IETF. <https://www.rfc-editor.org/rfc/rfc5280> (accessed July 17, 2026).
12. All source code: <https://github.com/fredteumer/jhu-ai-eng-course/tree/main/module8>